

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**ДВУПОСОЧНО ИНЖЕНЕРСТВО МЕЖДУ
ИЗХОДЕН КОД НА C++, UML ДИАГРАМИ НА
КЛАСОВЕ И РЕЛАЦИОННИ СХЕМИ НА
ДАННИ**

КУРСОВ ПРОЕКТ

по дисциплина „UML обектно-ориентирано проектиране“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Проф. д-р инж. Даниела Гоцева

София, **[TODO: година на предаване]**

СЪДЪРЖАНИЕ

УВОД	1
I. ПРЕДМЕТНА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД	3
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	5
2.1. Прочитане на изходния текст на C++	5
2.2. Формат на извеждане на диаграмата	7
2.3. Запис на съответствието клас към таблица	7
2.4. Език, система за построяване и външни зависимости	8
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	9
3.1. Модули и посока на зависимостите	9
3.2. Речник на системата и класификатори на модела	9
3.3. Извеждане на връзките от типа на члена	10
3.4. Шаблонни класове	11
3.5. Съответствие с релационна схема	11
3.6. Случаи на употреба и дейности	12
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	15
4.1. Обща структура	15
4.2. Четене на изходния текст	16
4.3. Заместващи заглавни файлове при извличането	16
4.4. Излъчватели	17
4.5. Пораждаща посока	17
4.6. Проверяващ модул	18
4.7. Модулни тестове	18
V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА	20
5.1. Среда на измерването	20
5.2. Първи експеримент: вярност на извличането	21
5.3. Втори експеримент: цена на извличането	21
5.4. Трети експеримент: запазване при пълен цикъл	22

5.5. Четвърти експеримент: откриваемост на внесено разминаване	22
VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	23
6.1. Вярност на извличането	23
6.2. Цена на извличането	25
6.3. Запазване на модела при пълен цикъл	26
6.4. Откриваемост на нарочно внесено разминаване	27
ЗАКЛЮЧЕНИЕ	29
ЛИТЕРАТУРА	31
ПРИЛОЖЕНИЯ	33
Приложение А. Формат на вътрешния модел	33
Приложение Б. Примерен вход и получените диаграми	37
Приложение В. Породена релационна схема	46
Приложение Г. Ключови места от изходния текст	50

УВОД

Обектно-ориентираният модел и програмният код, който го реализира, живеят в два различни носителя. Моделът се описва на UML и служи за разговор между хора, а кодът се описва на език за програмиране и служи за разговор с машина. Двата носителя се разминават почти веднага след като работата по проекта започне, защото кодът се променя всеки ден, а диаграмата се преначертава, когато някой се сети. Резултатът е познат: диаграма, на която вече никой не вярва, и код, чиято структура се възстановява само с четене.

Настоящият курсов проект разглежда това разминаване като инженерна задача с решение, а не като организационен навик. Ако структурата на кода може да бъде извлечена машинно, а UML моделът може да бъде записан в текстов вид, то преходът между двете посоки е преобразуване между два формални езика и подлежи на автоматизация. Терминологичната рамка за двете посоки е утвърдена от Chikofsky и Cross [1]: *обратно инженерство* е възстановяване на абстракция от реализация, а *право инженерство* е обратният преход от абстракция към реализация. Двете заедно образуват *двупосочно инженерство*, което е предметът на този проект.

Целта на проекта е да се проектира и реализира инструмент, наречен Blueprint, който извлича UML модел на класове от изходен код на C++, изобразява го в текстови нотации за диаграми, извежда от него релационна схема на данни и в обратната посока поражда заготовки на класове и схема по подаден модел, като допълнително отчита местата, на които моделът и кодът са се разминали.

За постигането на целта са формулирани следните **задачи**:

1. преглед на понятийния модел на UML, на градивните му елементи, на видовете връзки и на правилата и общите механизми, доколкото те засягат диаграмата на класове и нейното съответствие с изходен код;
2. анализ на съществуващите инструменти за извличане на структура от код и за двупосочно инженерство, и обосновка на избора на подход и на технологичен набор;
3. проектиране на вътрешния модел на системата, на границите между модулите и на правилата за преобразуване между клас, диаграма и релационна схема;
4. програмна реализация на извличането, на извеждането в двете текстови нотации, на пораждането на схема и заготовки и на проверката за разминаване;

5. изграждане на методика за експериментална проверка на верността на преобразуванията и на устойчивостта им при преминаване през пълен цикъл;
6. провеждане на експериментите и анализ на получените резултати.

Обхватът на работата е структурната част на обектно-ориентирания модел: класове, атрибути, операции, видимост, обхват, шаблонни класове и връзките между класовете, заедно със съответствието им на релационна схема. Диаграмите на случаите на употреба и на дейностите се разглеждат като контекст на модела и се използват за описание на самата система, но не се извличат от изходен код. Извън обхвата остават поведенческите диаграми, които изискват анализ на потока на управление, както и пълното семантично покритие на езика C++.

Структура на изложението. Раздел I излага предметната област и понятийния апарат на UML. Раздел II съпоставя възможните решения и обосновава избора. Раздел III представя проектирането на системата, а Раздел IV нейната програмна реализация. Раздел V описва методиката за проверка, а Раздел VI получените резултати. Заключението обобщава постигнатото и очертава по-нататъшната работа.

I. ПРЕДМЕТНА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

UML е графичен език за визуализиране, специфициране, конструиране и документиране на артефактите на една програмна система. Той не е метод и не предписва процес на разработка, а речник и правила за съчетаване на думите от този речник. Тази разлика е съществена за настоящия проект: щом езикът е формален, а не илюстративен, той има недвусмислена абстрактна структура и следователно подлежи на машинна обработка. Официалната спецификация на езика [2] задава именно тази структура под формата на метамодел, а ръководствата на авторите на езика [3, 4] го излагат в приложим за проектант вид.

Понятиятният модел на UML стъпва на три групи съставки. Първата са *градивните елементи*: структурни (клас, интерфейс, компонент, възел), поведенчески (взаимодействие, състояние) и групиращи (пакет), заедно с пояснителния елемент (бележка). Втората са *връзките*: зависимост, асоциация, обобщение и реализация, като асоциацията има отделни разновидности за агрегация и композиция, различаващи се по това дали частта може да съществува без цялото. Третата са *правилата и общите механизми*: спецификации, украшения, общи разделения и *механизмите за разширяване*, тоест ограничения, етикетираны стойности и стереотипи. Настоящият проект се интересува непосредствено от последните, защото именно чрез стереотип и етикетирана стойност се обозначава кой клас участва в релационната схема и как.

Диаграмата на класове е основната структурна диаграма и е носител на речника на системата. Класификаторът в нея се описва с име, атрибути, операции и отговорности, като видимостта и обхватът се отбелязват върху всеки член. Практиката за откриване на този речник е по-стара от самия UML: картите клас, отговорност и сътрудничество, познати като CRC карти, са предложени от Beck и Cunningham [5] точно като начин класовете да бъдат намерени преди да бъдат нарисувани. За компактно въведение в приложната част на езика е използван и [6].

Отношението между обектния модел и релационния модел на данните е самостоятелна тема със собствена литература. Релационната схема се описва чрез таблици, ключове и връзки между тях и има свой понятиен предшественик в модела същност и връзка на Chen [7]. Съответствието между клас и таблица не е взаимно еднозначно: наследяването няма пряк аналог в релационния модел, а асоциацията се изразява чрез

външен ключ или чрез свързваща таблица според кратността. Систематичен преглед на утвърдените стратегии за това съответствие дава [8]. Тези стратегии са непосредствената основа на правото инженерство в настоящия проект.

Мястото на двупосочното инженерство в жизнения цикъл на програмния продукт следва таксономията на Chikofsky и Cross [1]. Обратното инженерство поддържа документацията вярна, без да изисква дисциплина от разработчика, и е приложимо и към чужд код. Правото инженерство пренася решение, взето на нивото на модела, надолу до реализацията, без ръчно преписване. Стойността на двете заедно е в третото им следствие: щом и двете посоки са автоматизирани, разминаването между модел и код става измеримо, а не просто подозирано. **[TODO: при разширяване на раздела да се добави преглед на инструментите за проверка на съответствие между модел и код]**

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

2.1. Прочитане на изходния текст на C++

Изборът на начин за прочитане на изходния текст предопределя горната граница на всичко останало. Разгледани са четири възможности. Първата е анализ с регулярни изрази върху текста на файловете, който се пише за часове, но се проваля при шаблони, пространства от имена, макроси и всяко условно компилиране, тоест точно там, където реалният код е интересен. Втората е използване на съществуващ инструмент за документация, който вече разбира езика, и извличане на модела от неговия изход. Третата е свързване с libclang [9], тоест със самия компилаторен фронтенд през програмен интерфейс на C. Четвъртата е *изпълнение на компилатора като външен процес* с ключа `-Xclang -ast-dump=json` и разбор на изведеното от него абстрактно синтактично дърво в JSON [10, 11].

Първите две отпадат по същество. Текстовият анализ няма достъп до разрешените имена и следователно не може да отговори на въпроса дали `Wheel` в един файл и `Wheel` в друг са един и същ клас. Doxygen с Graphviz [12, 13] е най-близкият утвърден инструмент и наистина чете C++ и рисува диаграми на наследяване и на сътрудничество. Три обстоятелства обаче го правят негоден за целта тук. Диаграмите му са изображения, предназначени за четене от човек, а не модел, върху който може да се разсъждава машинно. Той не различава разновидностите на асоциацията по семантика на собствеността, а само отбелязва наличието на връзка. И най-важното, той няма права посока: от неговия изход не се поражда нито код, нито схема. Тоест той решава едната половина от задачата и по устройство не може да реши другата.

Изборът между третата и четвъртата възможност е същинският. И двете използват един и същ фронтенд и дават един и същ семантичен резултат: разрешени имена, типове след заместване на шаблонните аргументи и точно местоположение на всяка декларация. Разликата е изцяло в начина на свързване и е обобщена в Табл. 1.

Таблица 1. Съпоставка на начините за достъп до компилаторния фронтенд

Признак	Свързване с libclang	Изпълнение на компилатора и разбор на JSON
Зависимост при построяване	заглавни файлове и библиотека на LLVM	няма, нужен е само компилатор за C++20 и CMake
Зависимост при изпълнение	същата библиотека, със съпадаща версия	двоичният файл clang++, без изисквания към версията
Двоична съвместимост	обвързва проекта с ABI на конкретно издание на LLVM	няма ABI, границата е текст
Построяване на чужда машина	изисква пакетен управител или ръчна инсталация	работи веднага
Обхват без фронтенд	нищо не се построява	всичко освен извличането се построява и работи
Цена на прехода	извикване на функция	създаване на процес и разбор на текст, измерено в Раздел VI
Пълнота на данните	целият вътрешен модел на фронтенда	само това, което изводът съдържа, вж. по-долу

Приета е четвъртата възможност. Решаващият довод е зависимостта при построяване. Проектът трябва да се построява на чиста машина само с компилатор за C++20 и CMake, а свързването с libclang би изисквало инсталация на LLVM със заглавните файлове за разработка още при конфигурирането. При изпълнение на компилатора като външен процес фронтендът е вход на инструмента, а не част от него: без него не работи само извличането, докато излъчването на диаграми, пораждането на схема, пораждането на заготовки и проверката за разминаване продължават да работят. Границата между двата процеса е текст, така че отпадат и съпадението на версии, и двоичната съвместимост.

Изборът има и цена, която е измерена, а не премълчана. Създаването на процес и разборът на текста струват време, което при свързване с библиотека не се плаща. Освен това изводът в JSON не съдържа всичко, което вътрешният модел на фронтенда съдържа: установено е при проверка, че *аргументите на атрибутите* се губят, тоест `AnnotateAttr` се извежда като възел с местоположение, но без низа, който е бил подаден на атрибута.

Това обстоятелство пряко определя как се записва съответствието клас към таблица и е разгледано по-долу.

2.2. Формат на извеждане на диаграмата

Възможностите са пряко рисуване във векторен формат, извеждане в стандартизирания XML обмен за UML модели и извеждане в текстова нотация за диаграми. Прякото рисуване изисква реализация на алгоритъм за подреждане на графа и измества фокуса на проекта от моделирането към геометрията. XML е точен и стандартен, но е нечетим за човек и е полезен само ако насрещният инструмент вече го приема. Избрано е извеждане в две текстови нотации, PlantUML [14] и Mermaid [15], защото двете са четими в необработен вид, версионират се като текст и се изобразяват от съществуващи изобразители. Двете, а не една, защото различията между тях са проверка, че вътрешният модел е независим от изходния формат. Тази проверка не е формална: тя откри реална загуба, отчетена в Раздел VI.

2.3. Запис на съответствието клас към таблица

Разгледани са външен описателен файл, извеждане по съглашение за именуване и вграждане на описанието в самия код. Външният файл се разминава с кода по същата причина, по която се разминава и диаграмата. Съглашението за именуване е тихо и се чути при първото изключение. Избрано е описанието да живее в кода, което на нивото на модела се изобразява като UML стереотип и етикетирани стойности. Така механизмът за разширяване на UML и механизмът за аотиране на кода съвпадат по предназначение и преходът между тях е пряк.

Остава въпросът какъв да бъде носителът в кода. Естественият избор са атрибутите на C++20 [16], но той е неприложим по причина, установена с проверка, а не предположена: изводът на фронтенда в JSON записва атрибута като възел с диапазон в изходния текст и **без неговите аргументи**, тоест низът в `[[clang::annotate("table=books")]]` изобщо не достига до четеца. Приет е втори носител, `static constexpr` член с инициализатор, чиято стойност се извежда изцяло, включително низовият литерал. Изборът е стандартен C++20, не струва нищо по време на изпълнение, чете се от човек, отварящ заглавния файл, и не зависи от разширение на конкретен компилатор. Цената му е, че анотацията изглежда като член на класа; четецът разпознава представката `br_` и я премества върху

класификатора като стереотип или етикетирана стойност, така че тя никога не се появява като атрибут в модела.

2.4. Език, система за построяване и външни зависимости

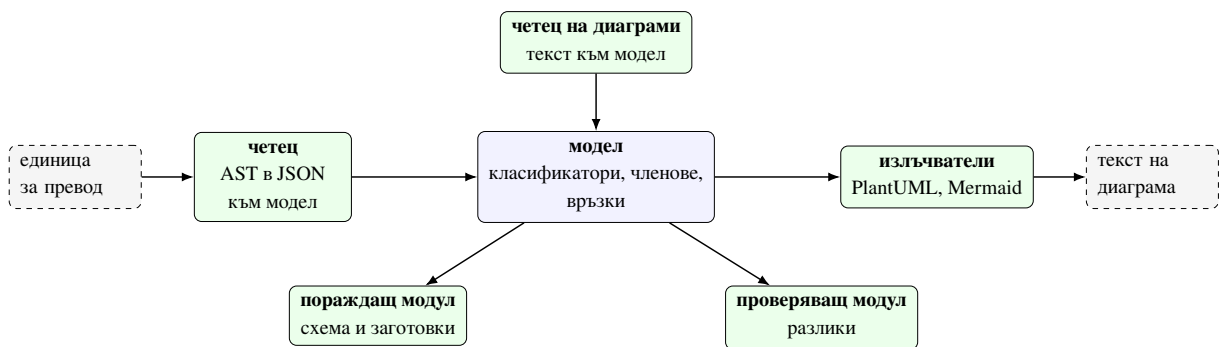
C++20 е приет за език на реализацията, защото инструментът работи върху C++ и разработчикът му остава в същата среда, и защото по-ранна работа на същия автор върху съпоставяне на класове и релационни схеми по време на компилация е решавала сродна задача от другата ѝ страна [17]. За система за построяване е приет CMake.

Приетото ограничение за външните зависимости е по-строго от обичайното: *нула задължителни външни библиотеки*. То не е вкус, а следствие от предназначението. Хранилището е публично и непознат човек трябва да може да изпълни `cmake -S . -B build` и `cmake --build build` и това да проработи от първия път. Построяване, което изисква пакетен управител, е построяване, което никой не прави. Следствията са три и всяко е съзнателно: за модулно тестване не е използвана външна рамка, а собствен изпълнител от около сто реда, свързан с CTest чрез `add_test`; за измерване на време не е използвана външна библиотека, а `std::chrono::steady_clock`; и за разбор на JSON е написан собствен четец, чийто обхват е точно това, което изводът на фронтенда изисква. Общо тези три решения добавят по-малко код, отколкото би добавила настройката на пакетен управител, и премахват изцяло въпроса дали проектът ще се построи другаде.

III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

3.1. Модули и посока на зависимостите

Системата е разделена на пет модула с еднопосочни зависимости. *Четецът* обхожда единиците за превод и произвежда вътрешен модел. *Моделът* е независим от езика на входа и от формата на изхода и е единственото място, където се пази структурата. *Излъчвателите* превръщат модела в текст на избраната нотация. *Пораждащият модул* превръща модела в релационна схема и в заготовки на класове. *Проверяващият модул* съпоставя два модела и съобщава разликите между тях. Зависимостите сочат само към модела, така че нито един излъчвател не знае за съществуването на четеща (Фиг. 1).



Фигура 1. Модули и посока на зависимостите. Всяка стрелка сочи натам, накъдето тече информация, и нито един модул не зависи от друг освен от модела

Правилото се проверява механично, а не се пази по споразумение: заглавните файлове на излъчвателите, на пораждащия и на проверяващия модул включват `model.hpp` и не включват `reader.hpp`. Ползата е практическа. Добавянето на трета нотация не докосва извличането, а промяна в извличането се отразява едновременно във всички изходни формати.

3.2. Речник на системата и класификатори на модела

Речникът на системата е получен по метода на CRC картите [5]: за всяка предполагаема отговорност е записан носителят ѝ и сътрудниците му, след което картите без собствено поведение са отпаднали или са се слели със съседите си. Така са отделени класификаторите на модела. Всеки клас в модела носи име, квалифицирано с пространството от имена, вид на класификатора, списък от шаблонни параметри, списък от атрибути, списък от операции и списък от приложените върху него стереотипи и

етикетирани стойности. Атрибутът носи име, тип, видимост, обхват, кратност и собствени етикетирани стойности. Операцията носи име, тип на резултата, списък от параметри, видимост, обхват и признаци за виртуалност, чистота и константност. Видимостта и обхватът се отбелязват по правилата на UML, като статичните членове се маркират като членове с обхват на класификатора, а не на екземпляра.

Видовете класификатори са пет: клас, структура, интерфейс, изброен тип и шаблон. Интерфейсът не се чете от изходния текст, защото C++ няма такава ключова дума, а се *извежда*: класификатор без атрибути, чиито операции са само чисти виртуални, е интерфейс по определението на UML. Изводът се прави след обхождането, върху модела, а не върху синтаксиса, което е и мястото, където му е мястото.

3.3. Извеждане на връзките от типа на члена

Връзките между класовете се извеждат от типовете на членовете, а не се обявяват отделно. Обобщението следва пряко от списъка на базовите класове, заедно с достъпа на всяка база и с признака за виртуално наследяване. Асоциацията се установява, когато член на един клас се отнася към друг клас от модела. Разграничението между агрегация и композиция се прави по това дали обектът се притежава и дали притежанието е изключително. Зависимостта се извежда от употребата на тип в подпис на операция, когато между двата класификатора няма по-силна връзка. Реализацията свързва специализация на шаблон с общия шаблон. Правилата са изнесени в таблица, за да бъдат ревизуеми поотделно (Табл. 2).

Таблица 2. Съответствие между изписването на члена и вида на връзката

Изписване на члена	Притежание	Връзка	Кратност
T value	изключително	композиция	1
std::unique_ptr<T>	изключително	композиция	0..1
std::optional<T>	изключително	композиция	0..1
std::shared_ptr<T>	споделено	агрегация	0..1
std::weak_ptr<T>	споделено	агрегация	0..1
T*	няма	асоциация	0..1
T&	няма	асоциация	1
контейнер от някое от горните	като елемента	като елемента	*
тип само в подпис на операция	няма	зависимост	

Прочитът следва UML буквално. Композиция означава, че частта загива заедно с цялото, затова член по стойност или член в указател с изключително притежание е композиция. `shared_ptr` задържа частта жива и след цялото, което е агрегация. Обикновен указател или отправка не твърдят нищо за времето на живот, което оставя проста асоциация. При контейнер кратността става *, а видът на връзката се определя от елемента: `std::vector<T*>` е асоциация с много, а `std::vector<std::unique_ptr<T>>` е композиция с много. При асоциативен контейнер се моделира типът на стойността, не на ключа. Член, чийто тип не е класификатор от модела, остава обикновен атрибут и не поражда връзка.

3.4. Шаблонни класове

Шаблонните класове се записват като параметризирани класификатори, като списъкът на параметрите е част от класификатора. Пълните и частичните специализации се записват като отделни класификатори със връзка на реализация към общия шаблон, а частичната специализация носи и ограничение върху параметрите като етикетирана стойност. Изборът е съзнателен: разгръщането на всяка инстанция в отделен клас затрупва диаграмата с еднотипни възли и заличава самата параметризация, която е носител на смисъла.

По същата причина *инстанциациите, породени от употребата*, а не от декларацията, не влизат в модела. Фронтендът ги извежда наравно с декларираните, но те са свойство на конкретния превод, а не на проектното решение: добавен ред в тяло на функция би добавил клас в диаграмата. Моделът съдържа само това, което е декларирано.

3.5. Съответствие с релационна схема

Съответствието е описано с механизмите за разширяване на UML. Клас, обозначен със стереотипа за постоянно съхранение, поражда таблица, всеки негов атрибут със скаларен тип поражда колона, а етикетиранияте стойности задават име на таблицата, име и тип на колоната и участие в първичния ключ. Клас без обозначен първичен ключ получава сурогатен ключ `id`, като това решение се съобщава изрично, а не се прилага мълчаливо.

Асоциацията с кратност едно или нула до едно поражда външен ключ в притежаващата страна. Асоциацията с кратност много поражда свързваща таблица. Изборът за много е съзнателен и се основава на това, че диаграмата показва само единия

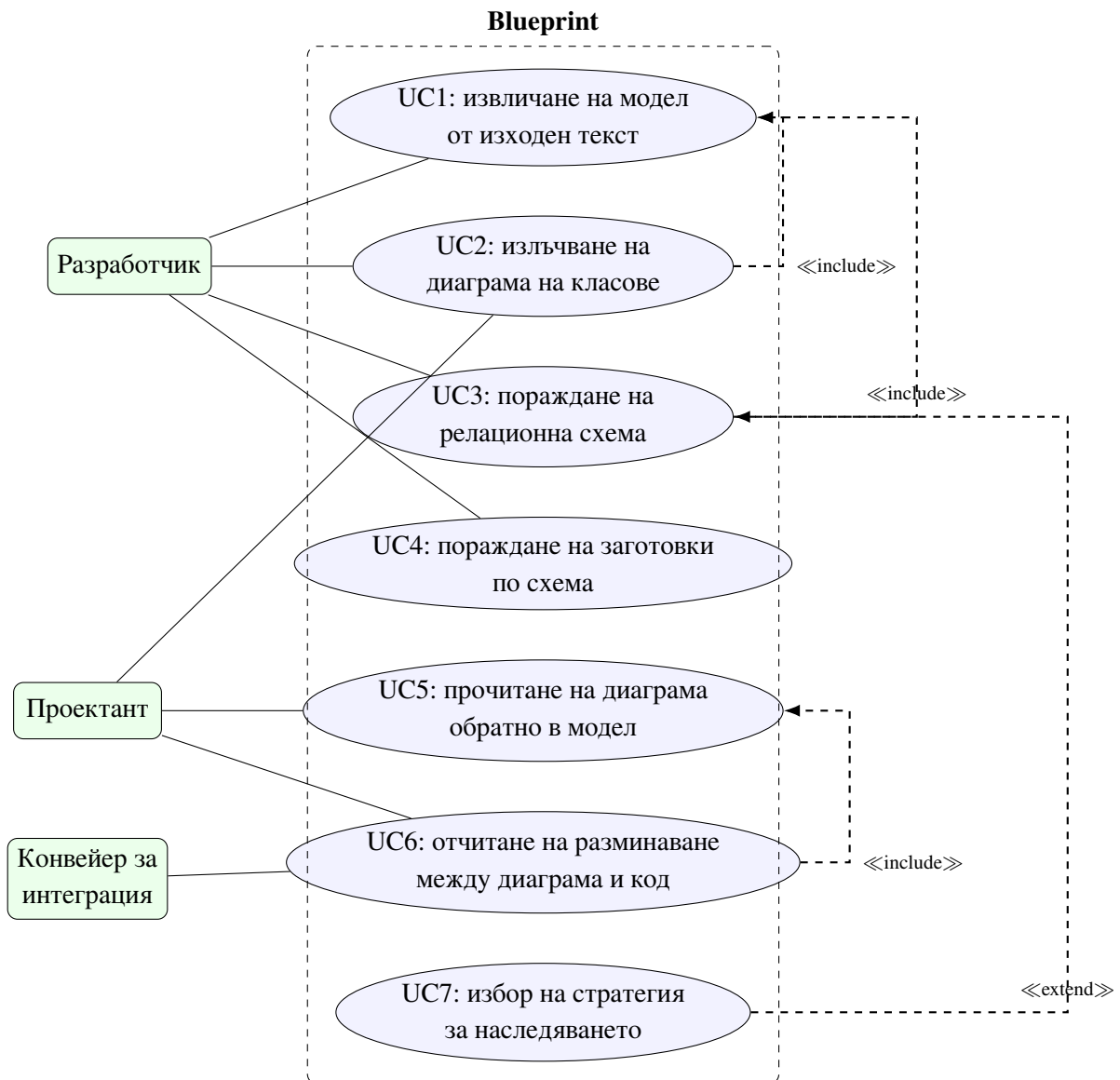
край: от нея не се вижда дали връзката е едно към много или много към много, а свързващата таблица е вярна и в двата случая, докато външният ключ е верен само в единия.

За наследяването са реализирани и трите утвърдени стратегии [8]: една таблица за цялата йерархия с колона разграничител, таблица на всеки клас с ключ, който е и външен ключ към базата, и таблица на всеки конкретен клас с копирани надолу наследени колони. По подразбиране се прилага стратегията *една таблица за йерархията*, защото тя е единствената, при която извличането на един обект не изисква съединяване и която следователно не изненадва читателя на породената схема. Стратегията не е зазидана в инструмента: тя се избира с етикетирани стойност върху базовия клас, а подадената от командния ред стойност служи само когато такава етикетирани стойност няма.

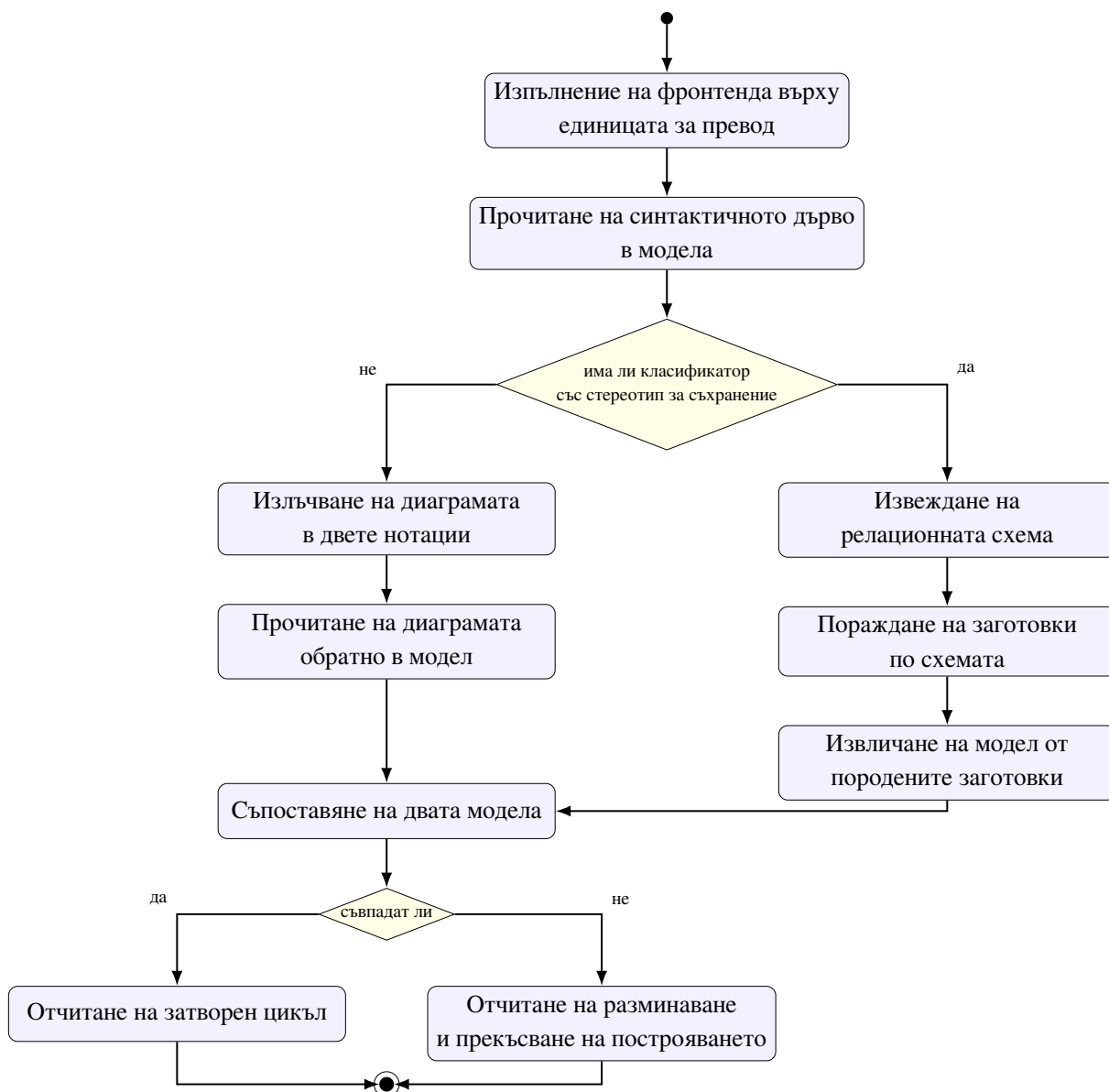
3.6. Случаи на употреба и дейности

Използването на системата е описано с диаграма на случаите на употреба (Фиг. 2), а редът на операциите при преминаване през пълен цикъл с диаграма на дейностите (Фиг. 3). Двете диаграми са част от документацията на самия проект и служат едновременно като пример за нотацията, разгледана в Раздел I.

Нито едната от двете не може да бъде извлечена от изходен текст. Случаят на употреба е твърдение за намерение, а дейността е твърдение за поток на управление, който структурата на кода не носи. Затова и двете се описват на ръка в малък текстов вид и се превеждат към същите две нотации като диаграмата на класове. Така правилото „един модел, много изходни формати“ важи и за поведенческата част, без да се твърди, че тя се възстановява от кода.



Фигура 2. Случаи на употреба. Изходният текст на модела е в `examples/blueprint.usecase` и се превежда към двете нотации с `blueprint usecase`



Фигура 3. Дейности при преминаване през пълен цикъл. Изходният текст на модела е в examples/roundtrip.activity и се превежда с blueprint activity

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

4.1. Обща структура

Реализацията е на C++20 [16] и се построява с CMake. Проектът е разделен на библиотека, която съдържа цялата логика, и изпълним файл, който е тънък слой над нея за работа от команден ред. Разделението не е украса: модулните тестове се свързват срещу библиотеката и не минават през командния ред, така че проверката на поведението е независима от начина на извикване. Публичните заглавни файлове живеят в `include/blueprint/`, реализацията в `src/`, а тестовете в `tests/`. Всеки модул от Раздел III има точно един заглавен файл и точно един файл с реализация (Табл. 3).

Таблица 3. Файлове на реализацията и отговорността на всеки

Файл	Отговорност
<code>json.{hpp,cpp}</code>	Документен модел, четец и записвач на JSON. Съществува, защото изводът на фронтенда е JSON, а проектът не приема външни зависимости.
<code>model.{hpp,cpp}</code>	Вътрешният модел и записът му във външен вид. Единственото място, където се пази структура.
<code>reader.{hpp,cpp}</code>	Обхождане на синтактичното дърво, извеждане на връзките, изпълнение на компилатора.
<code>emit.{hpp,cpp}</code>	Общият обход и двата излъчвателя.
<code>diagram.{hpp,cpp}</code>	Прочитане на диаграма обратно в модел, за двете нотации.
<code>schema.{hpp,cpp}</code>	Модел към релационна схема, схема към DDL и обратно, схема към заготовки на класове.
<code>diff.{hpp,cpp}</code>	Съпоставяне на два модела и отчитане на разликите.
<code>behavior.{hpp,cpp}</code>	Модели и излъчватели за случаите на употреба и за дейностите.
<code>annotations.hpp</code>	Макросите за аотиране, само дефиниции на препроцесора.
<code>io.{hpp,cpp}</code>	Четене и запис на файл, за да се отказват еднакво навсякъде.
<code>main.cpp</code>	Команден ред. Не взема решения, само подрежда извиквания.

4.2. Четене на изходния текст

Четенето става чрез изпълнение на компилатора като външен процес и разбор на изведеното от него синтактично дърво в JSON [10]. Съставянето на командата е отделено в `build_clang_command`, което я прави проверима без да се изпълнява процес, а разборът на резултата е отделен в `read_ast_json`, което е чиста функция без достъп до файловата система. Разделянето не е формално: то позволява целият четец да се тества върху записан образец на дърво, без инсталиран компилатор.

Три подробности от обхождането заслужават коментар, защото и трите са места, на които наивната реализация мълчаливо произвежда грешен модел.

Проследяване на текущия файл. Фронтендът извежда полето с името на файла *само когато то се променя*, а следващите взли го наследяват. Затова текущият файл се пренася през цялото обхождане в реда на документа. Пропускането на тази подробност е разликата между модел на проекта и модел на всеки системен заглавен файл, който проектът включва.

Отсяване на подразбиращите се членове. Фронтендът извежда и членовете, които самият той е породил: подразбиращите се конструктори, операторите за присвояване, деструкторът, както и вграденото име на класа в самия него. Всички те носят признака `isImplicit` и се отхвърлят. Без това всеки клас в диаграмата получава по пет операции, които никой не е писал.

Видимост по секции. Видимостта на един член не се извежда като негово поле, а следва от предхождащия го спецификатор за достъп. Затова тя се води при обхождането, започвайки от `private` за `class` и от `public` за `struct`.

Разрешаването на имената до връзки се извършва *след* обхождането, когато всички класове вече са известни, защото типът на член може да сочи към клас, деклариран по-късно в същата единица за превод. Съответствието на име се прави първо по напълно квалифицирано име, а ако то не съвпадне, по последната съставка. Двусмислено име не се разрешава до предположение, а остава неразрешено и не поражда връзка.

4.3. Заместващи заглавни файлове при извличането

Извличането се изпълнява с `-nostdinc++` и с папката `stdstub/` като системна папка за включване. Тя съдържа заглавни файлове само с декларации, заместващи тези

от стандартната библиотека, които заглавните файлове на проекта включват. Причината е измерена, а не стилистична: една единица за превод, която включва действителните `<memory>`, `<vector>` и `<string>`, извежда около 280 MB JSON, почти изцяло вътрешности на стандартната библиотека, които никоя диаграма на класове не би показала. Със заместващите файлове същата единица извежда под мегабайт. Те никога не се компилират в инструмента и никога не се свързват, а извличането се изпълнява с `-fsyntax-only`.

4.4. Излъчватели

Излъчвателите в двете нотации [14, 15] споделят един обход на модела и се различават само по низовете, които произвеждат за всеки елемент. Общата част е изнесена в базов клас с виртуални методи за отделните елементи, а конкретните излъчватели предефинират само тях. Редът на елементите и решението кое си заслужава да бъде излъчено живеят в базовия клас, веднъж.

Това е и мястото, където се вижда цената на разликите между нотациите. Mermaid чете ъгловите скоби като свой собствен ограничител за параметризация и използва тилда, така че всяко изписване на тип се превежда на излизане. Нито една от двете нотации няма място за етикетирана стойност или за списък на шаблонните параметри, затова те пътуват в *бележка*. Бележката е механизмът на самия UML за пояснение върху елемент, тоест това не е скрит канал, а предвиденият. Един признак остава без превод: Mermaid отбелязва операция като абстрактна или като статична и няма нищо за операция, която е виртуална, но не е чиста. Загубата е измерена, а не премълчана, и е отчетена в Раздел VI.

Четецът на диаграми приема диалекта, който самият Blueprint произвежда. Ограничението е съзнателно и е заявено като такова: целта на четенето е да затвори цикъла и да даде на проверяващия модул модел на документираното решение, а не да приеме всяка диаграма, която човек би могъл да нарисова.

4.5. Пораждаща посока

Пораждащата посока е реализирана като две отделни преобразувания над един и същ модел. Първото произвежда описание на релационна схема, второто заготовки на класове. И трите стратегии за наследяване са реализирани, като изборът се прави от етикетирана стойност върху базовия клас и само при липсата ѝ от подадената от командния ред стойност. При стратегията с една таблица всяка колона, дошла от наследник, се

записва като допускаща празна стойност, защото ред от друг наследник я оставя празна, и се добавя колона разграничител. При стратегията с таблица на клас първичният ключ на наследника е и външен ключ към базата. При стратегията с таблица на конкретен клас абстрактният клас не получава таблица, а наследените колони се копират надолу.

Породеният DDL носи името на класа в коментар пред всяка таблица и името на атрибута в коментар след всяка колона, чието име се различава от неговото. Това не е украса. Възстановяването на име на клас от име на таблица би означавало отгатване на английско единствено число, а отгатването е точно това, което един пълен цикъл не бива да съдържа.

Заготовките се пораждаат само за декларации, тоест инструментът не пише тела на операции и не докосва съществуващ файл, който вече съдържа реализация. Породеният заглавен файл носи и макросите за аотиране, така че повторното извличане от него дава съпоставим модел, с което цикълът се затваря. Че породеният файл е действителен C++, се проверява в демонстрацията чрез `clang -fsyntax-only` върху него, а не се твърди.

4.6. Проверяващ модул

Проверяващият модул сравнява модел, извлечен от кода, с модел, прочетен от диаграма, и извежда разликите като списък. Видовете разлика са девет: добавен, премахнат и променен, поотделно за класификатор, член и връзка. Съответствието на връзките се прави по двата края и по етикета, така че промяна на вида или на кратността се отчита като промяна, а не като премахване плюс добавяне. Разликата е съществена: два реда вместо един за една и съща промяна биха направили отчета по-шумен точно там, където той трябва да е най-полезен.

Посоката на думите е фиксирана: първият подаден модел е кодът, вторият е решението, така че клас, който е в кода, но не е в диаграмата, е *добавен*, а обратното е *премахнат*. Процесът завършва с код 0 при съвпадение и с код 1 при разлика, което е и условието проверката да може да прекъсне построяване, вместо да чака някой да я прочете.

4.7. Модулни тестове

Модулните тестове са 59 в 7 групи и не използват външна рамка. Изпълнителят е един заглавен файл с макроси за твърдение и регистър, попълван при статично

инициализиране; групите се регистрират в CTest поотделно с `add_test`, така че CTest съобщава по един ред на група, а всяка група изброява своите случаи. Тестовите на четеща работят върху записан образец на синтактично дърво, в който присъстват само полетата, които четещът действително чете, тоест образецът е едновременно вход на теста и заявление какво четещът изисква от фронтенда.

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА

Проверката отговаря на четири отделни въпроса и затова е разделена на четири експеримента. Първият пита дали извлеченият модел отговаря на изходния текст. Вторият пита колко струва извличането и как цената расте с размера на входа. Третият пита дали преминаването през пълен цикъл запазва модела. Четвъртият пита дали проверяващият модул открива разминаване, което е внесено нарочно. Въпросите са различни и смесването им в едно число би скрило точно това, което трябва да бъде видно.

5.1. Среда на измерването

Средата се записва изцяло, тъй като резултатите зависят от версията на фронтенда. Тя се извежда от самия измервателен скрипт, а не се преписва на ръка (Табл. 4).

Таблица 4. Среда, в която са проведени измерванията

Съставка	Стойност
Операционна система	Windows 10.0.26200
Процесор	AMD Ryzen 5 3600, 6 ядра, 12 логически ядра
Компилятор при построяване	GNU 15.2.0 (MinGW-w64, x86_64-posix-seh)
Фронтенд при извличане	clang 22.1.0
Система за построяване	CMake 4.3.2, генератор Ninja 1.13.2
Режим на построяване	Release
Стандарт на езика	C++20

Всяко измерване се извършва след чисто построяване. Времената се повтарят по три пъти и се съобщава *най-малкото* от повторенията, а не средното: по-бавните изпълнения се различават по планиране на процесите на операционната система, което не е свойство на инструмента. Всички измервания се възпроизвеждат с една команда, `smake --build build --target measure`, а четвъртият експеримент с `smake --build build --target divergence`. Формата на резултатите е фиксирана предварително и е описана в Раздел VI, за да не бъде избрана след като числата вече са известни.

5.2. Първи експеримент: вярност на извличането

Първият експеримент използва набор от езикови конструкции, всяка от които се проверява поотделно: наследяване с достъп на базата, виртуално наследяване, множествено наследяване, член по стойност, член в притежаващ указател, член в споделен указател, член указател без притежание, член отправка, контейнер от всяка от тези форми, асоциативен контейнер, статичен член, чиста виртуална операция, шаблонен клас, влагане в пространство от имена, подразбиращи се членове, породени от компилатора, и анотация за постоянно съхранение.

За всяка конструкция очакваният модел е записан на ръка *преди* инструментът да бъде пуснат върху нея, като образец на синтактично дърво, съдържащ само полетата, които четецът чете. Сравнението е между очакван и получен модел, а не между два изхода на самия инструмент, за да не се проверява инструментът сам срещу себе си. Носителят на този експеримент са модулните тестове: всеки от тях е една конструкция с един записан очакван резултат, а изпълнението им е `cctest --test-dir build --output-on-failure`.

Отделно от това се проверява, че породените от инструмента заготовки на класове са действителен C++, като върху тях се пуска самият фронтенд с `-fsyntax-only`. Това е твърдение, което не бива да бъде прието на доверие, тъй като пораждащата посока пише код, който никой човек не е чел.

5.3. Втори експеримент: цена на извличането

Вторият експеримент измерва как цената на извличането зависи от размера на единицата за превод. Вховете са два вида. Първият е поредица от породени единици за превод с нарастващ брой класове, като всеки клас съдържа по един представител на всяка форма на член от Табл. 2, за да се мери действителна работа, а не файл с празни класове. Вторият вид са двата действителни входа, които хранилището съдържа: анотираният пример и единицата, която включва всички публични заглавни файлове на самия инструмент.

Измерват се поотделно три времена: изпълнението на фронтенда до готов JSON, разборът на този JSON и обхождането му до модел. Разделянето е съществено, защото трите се променят различно с размера на входа и едно общо число би скрило коя част расте. Записват се и размерът на изходния текст, размерът на изведеното дърво, броят на възлите

в него, размерът на записания модел и броят на елементите му, така че съотношението модел към изходен текст да бъде видимо, а не разказано.

5.4. Трети експеримент: запазване при пълен цикъл

Трети експеримент проверява устойчивостта при пълен цикъл. Изходен текст се превръща в модел, моделът в диаграма, диаграмата обратно в модел, и двата модела се сравняват. Ако преобразуванията са без загуба на информация, двата модела съвпадат. Същото се прави и по другата пораждаща посока: модел, схема, заготовки на класове, модел. Този експеримент е по-силен от първия, защото не изисква очакван резултат да бъде написан на ръка и следователно може да бъде проведен върху истински код. Като вход се използват аотираният пример и самият изходен текст на проекта, което го прави и проверка на инструмента върху него самия.

Показателят е дял на *запазените елементи*, а не единица минус дял на разликите. Разликата между двата показателя е съществена: когато цял класификатор не се върне, членовете му изобщо не се сравняват, и брой на разликите би нарекъл това малка загуба. Затова се броят поелементно класификаторите, атрибутите, операциите и връзките, които се появяват отново непроменени.

5.5. Четвърти експеримент: откриваемост на внесено разминаване

Четвъртият експеримент внася разминаване нарочно. От вече съгласувана двойка код и диаграма се променя само едната страна, по една промяна наведнъж: премахва се атрибут, добавя се операция, сменя се видимост, премахва се базов клас, сменя се кратност. Преди всяка промяна се проверява, че недокоснатата двойка съвпада, тъй като иначе нищо след това не значи нищо. След всяка промяна проверяващият модул се пуска и се записва дали очакваното съобщение е налице и колко съобщения са изведени общо.

Оценяването е по два показателя, дял на откритите внесени разминавания и брой на съобщенията извън очакваното. Двата показателя се докладват поотделно, защото инструмент, който съобщава за всичко, постига идеална откриваемост и е безполезен. Съобщение извън очакваното не се брой автоматично за грешно: то се разглежда поотделно и се посочва дали е следствие от същия факт, или е шум.

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

Резултатите се представят в реда на четирите експеримента от Раздел V. Всяка таблица е придружена от коментар за това какво следва от нея и какво не следва. Разминаванията, които останат неразрешени, се изброяват поименно, а не се обобщават в дял, защото един необяснен случай е по-полезен от една закръглена стойност. Всяко число тук е измерено на средата от Табл. 4 и се възпроизвежда с командите, посочени в Раздел V. Размерът на изведеното дърво и размерът на записания модел съдържат абсолютните пътища на входните файлове, затова тези две колони се възпроизвеждат точно при същия път и се отменят с постоянна стойност при друг. Никоя друга стойност не зависи от пътя.

6.1. Вярност на извличането

Първият експеримент проверява верността на извличането по отделни езикови конструкции. Резултатът се записва по един ред на конструкция, с изричен признак за съвпадение с предварително записания очакван модел (Табл. 5). Изпълнението е `cstest --test-dir build --output-on-failure`, което съобщава 7 групи и 59 случая, като всички преминават.

Таблица 5. Вярност на извличането по езикови конструкции

Конструкция	Съвпадение	Бележка
Наследяване с достъп на базата	✓	достъпът се чете от полето за базата
Виртуално наследяване	✓	признакът се пази в модела
Множествено наследяване	✓	две обобщения от един класификатор
Член по стойност	✓	композиция, кратност 1
Член в притежаващ указател	✓	композиция, кратност 0..1
Член в споделен указател	✓	агрегация, кратност 0..1
Член указател без притежание	✓	асоциация, кратност 0..1
Член отправка	✓	асоциация, кратност 1
Контейнер от стойности	✓	композиция, кратност *
Контейнер от указатели	✓	асоциация, кратност *
Асоциативен контейнер	✓	модела се типът на стойността
Статичен член	✓	обхват на класификатора
Чиста виртуална операция	✓	класификаторът става абстрактен
Извеждане на интерфейс	✓	без атрибути и само чисти операции
Шаблонен клас	✓	параметрите влизат в класификатора
Влагане в пространство от имена	✓	квалифицираното име е пълно
Подразбиращи се членове	✓	отсяват се, не влизат в модела
Анотация за съхранение	✓	става стереотип и етикетирани стойности
Отсяване по файл на произход	✓	извън заявената папка не влиза
Породените заготовки са валиден C++	✓	clang -fsyntax-only ги приема

Таблицата казва, че всяка от изброените конструкции се извлича така, както е записано предварително. Тя не казва, че покритието на езика е пълно. Извън проверката остават приятелските декларации, вложените шаблони на шаблони, наследяването от специализация на шаблон и всичко, което препроцесорът е скрил при конкретния набор от ключове. Тези случаи не са проверени и затова не се твърди нищо за тях.

6.2. Цена на извличането

Вторият експеримент измерва цената на извличането спрямо размера на входа. Породените входове съдържат от 10 до 800 класа, всеки с по един представител на всяка форма на член. Съобщава се най-малкото от три повторения (Табл. 6).

Таблица 6. Цена на извличането спрямо размера на единицата за превод

Класове	Изм. текст [B]	Дърво [B]	Възли	Елементи	Фронтенд [s]	Разбор [s]	Обход [s]
10	3 339	554 127	3 956	158	0.0631	0.0027	0.0004
25	8 559	798 297	5 861	413	0.0688	0.0042	0.0010
50	17 259	1 206 503	9 036	838	0.0744	0.0067	0.0021
100	34 660	2 024 872	15 386	1 688	0.0932	0.0116	0.0055
200	70 360	3 662 672	28 086	3 388	0.1161	0.0215	0.0124
400	141 760	6 945 391	53 486	6 788	0.1790	0.0413	0.0323
800	284 560	13 524 380	104 286	13 588	0.2871	0.0815	0.0925

От таблицата следват три неща. Първо, размерът на изведеното дърво расте линейно спрямо изходния текст, но с коефициент около 48, тоест дървото е с почти два порядъка по-голямо от текста, който го е породил. Това е и причината заместващите заглавни файлове от Раздел IV да не са удобство, а условие: при действителната стандартна библиотека същият коефициент дава стотици мегабайти.

Второ, изпълнението на фронтенда е най-скъпата съставка при малък вход и остава такава до около 400 класа, но не расте пропорционално: при осемдесеткратно увеличение на изходния текст то нараства 4.5 пъти, защото значителна част от него е постоянна цена за създаване на процес и за прочитане на заместващите заглавни файлове. Разборът на JSON расте практически линейно спрямо размера на дървото, което е очакваното поведение за еднократен проход.

Трето и най-важно, обхождането до модел расте *по-бързо от линейното*: между 400 и 800 класа входът се удвоява, а времето се увеличава 2.9 пъти. Причината е известна и е в кода: разрешаването на име до класификатор търси последователно в списък от всички класификатори, тоест е квадратично спрямо техния брой. При 800 класа това вече е най-скъпата съставка от трите. Поправката е очевидна, речник вместо списък, и е записана като по-нататъшна работа, а не е приложена, защото при действителните входове разликата не е измерима.

Двата действителни входа са дадени отделно, тъй като те, а не породените, показват какво инструментът върши в употреба (Табл. 7).

Таблица 7. Двата действителни входа

Вход	Изм. текст [B]	Дърво [B]	Класи- фикатори	Връзки	Модел [B]	Общо [s]
Анотиран пример	3 069	551 549	6	8	6 549	0.066
Заглавни файлове на инструмента	612	2 952 078	35	30	33 366	0.108

Вторият ред заслужава пояснение. Изходният текст е 612 байта, защото файлът съдържа само включвания на деветте публични заглавни файла на инструмента; същинският вход е това, което те внасят. Записаният модел е около 33 KB срещу близо 3 MB дърво, тоест моделът е около 88 пъти по-малък от дървото, от което е получен. Това е количествената форма на твърдението, че моделът е абстракция: почти цялото дърво е синтаксис, който диаграмата не показва.

6.3. Запазване на модела при пълен цикъл

Третият експеримент проверява запазването на модела при пълен цикъл, поотделно за двете нотации и за пътя през релационната схема (Табл. 8).

Таблица 8. Запазване на модела при пълен цикъл

Вход	Път	Елементи	Запазени	Разлики	Дял
Анотиран пример	PlantUML	41	41	0	1.000
Анотиран пример	Mermaid	41	38	3	0.927
Анотиран пример	схема	41	12	27	0.293
Заглавни файлове	PlantUML	226	226	0	1.000
Заглавни файлове	Mermaid	226	225	1	0.996
Заглавни файлове	схема	226	0	65	0.000

Пътят през PlantUML е без загуба и за двата входа. Това не е самоочевидно и не беше вярно от самото начало: при първото измерване през двете нотации се губеха имената на безименните параметри, защото излъчвателят ги записваше като : тип и четецът ги прочиташе обратно като тип с име : тип. Дефектът беше открит именно от този експеримент, а не от преглед на кода, и е поправен.

Пътят през Mermaid губи по един елемент на всяка операция, която е виртуална, но не е чиста. Нотацията отбелязва операция като абстрактна или като статична и няма трети знак. Загубата е свойство на нотацията, не на модела, и не подлежи на поправка в инструмента: единственият начин да бъде избегната е признакът да пътува в бележка, което би направило диаграмата нечетима за човек заради три букви. При аотириания пример такива операции са три, при заглавните файлове една, откъдето и разликата между 0.927 и 0.996. Тоест делът зависи от това какъв дял от модела са виртуалните операции, а не от размера му.

Пътят през релационната схема губи най-много, и това е очакваното. Таблицата няма операции, няма видимост, няма шаблони и няма класове, които не са обозначени за постоянно съхранение. При аотириания пример се запазват 12 от 41 елемента, тоест атрибутите на обозначените класове и техните класификатори. При заглавните файлове на инструмента се запазва нула, защото нито един негов клас не е обозначен за съхранение, така че схемата е празна. Нулата не е дефект, а вярно измерване на въпроса „колко от този модел изобщо може да бъде изразено като таблици“, и отговорът за инструмент, който не пази нищо в база от данни, е нула.

Общият извод от таблицата е, че пълният цикъл е затворен през нотациите и е *частичен* през схемата. Той е достатъчен, за да се отговори на въпроса дали кодът и диаграмата се разминават, и не е достатъчен, за да се възстанови кодът от схемата. Второто и не се твърди.

6.4. Откриваемост на нарочно внесено разминаване

Четвъртият експеримент внася по една промяна наведнъж. Преди всяка промяна е проверено, че недокоснатата двойка съвпада (Табл. 9).

Таблица 9. Откриваемост на нарочно внесено разминаване

Вид на промяната	Очакван вид съобщение	Открита	Общо съобщения	Извън очакваното
Премахнат атрибут	добавен член	✓	1	0
Добавена операция	премахнат член	✓	1	0
Сменена видимост	променен член	✓	1	0
Премахнат базов клас	добавена връзка	✓	1	0
Сменена кратност	променена връзка	✓	2	1

И петте внесени разминавания са открити, тоест откриваемостта е 5 от 5. Четири от петте дават точно по едно съобщение. Петото дава две и заслужава да бъде разгледано поименно, вместо да бъде отчетено като шум.

При смяна на кратността от 0..1 на 1 проверяващият модул съобщава и за променена връзка, и за променен атрибут. Второто съобщение не е грешно: кратността се пази на две места в модела, върху връзката и върху атрибута, който я е породил, и промяната на диаграмата променя и двете. Тоест едно редактиране води до две верни съобщения, а не до едно вярно и едно фалшиво. Наблюдението обаче сочи истинско проектно решение, което си заслужава да бъде записано: кратността е *дублирана* в модела. Дублирането е удобно за излъчвателите, които показват кратността и на двете места, и е неудобно за проверяващия модул, който заради него брои една промяна два пъти. Отстраняването му е въпрос на избор кой от двата носителя е първичен, и е записано като по-нататъшна работа.

Дял на съобщенията, които не отговарят на действително разминаване, е нула за всичките пет случая. Това е важното число: инструмент, който съобщава за всичко, би постигнал същата откриваемост и би бил безполезен.

ЗАКЛЮЧЕНИЕ

В курсовия проект е проектиран и реализиран инструмент за двупосочно инженерство между изходен код на C++, UML диаграма на класове и релационна схема на данни. Приетият подход поставя в средата един вътрешен модел, независим както от езика на входа, така и от формата на изхода, и свежда всяко преобразуване до преход между този модел и една външна форма. Следствието е, че добавянето на нова нотация не засяга извличането, а промяна в извличането се отразява едновременно във всички изходни формати.

Предимствата на решението произтичат от избора на компилаторен фронтенд за прочитане на кода. Понятия като разрешено име, тип след заместване на шаблонните аргументи и собственост върху обект са достъпни точно защото анализът е семантичен, а не текстов. Това е и условието разграничението между агрегация и композиция да бъде правено по устройство, а не по предположение. Второто предимство е в начина на достъп до фронтенда: той се изпълнява като външен процес, а изводът му се разбира като текст, така че проектът няма нито една задължителна външна зависимост и се построява само с компилатор за C++20 и CMake. Ограниченията са в същата посока: инструментът вижда само това, което изводът на фронтенда съдържа, а при това извличане стандартната библиотека е заместена с декларации, тоест типове извън заявената папка остават непрозрачни атрибути, а не връзки.

Измереното от четирите експеримента дава следната картина. Всички 20 проверени езикови конструкции се извличат така, както очакваният модел е записан предварително, а породените от инструмента заготовки на класове са приети от компилатора. Цената на извличането е под 0.11 s за действителните входове, като изведеното синтактично дърво е около 48 пъти по-голямо от изходния текст, а полученият от него модел е около 88 пъти по-малък от дървото, тоест абстракцията е количествено измерима, а не само декларирана. Пълният цикъл през PlantUML е без загуба и за двата входа, през Mermaid запазва 0.996 от елементите при заглавните файлове на инструмента и 0.927 при аотириания пример, а през релационната схема запазва 0.293 при аотириания пример и нула при заглавните файлове. И петте нарочно внесени разминавания са открити, при нула съобщения, които не отговарят на действително разминаване.

Три от тези числа заслужават трактовка, а не само отчитане. Първо, разликата

между 1.000 и 0.996 не е дефект на модела, а свойство на нотацията: Mermaid отбелязва операция като абстрактна или като статична и няма трети знак за операция, която е виртуална, но не е чиста. Именно това е ползата от два излъчвателя вместо един. Второ, нулата през релационната схема при заглавните файлове на инструмента е вярна: нито един клас на инструмента не е обозначен за постоянно съхранение, така че въпросът „колко от този модел може да бъде изразено като таблици“ има отговор нула. Трето, обхождането до модел расте по-бързо от линейното, защото разрешаването на име търси последователно в списък от всички класификатори; при 800 класа това вече е най-скъпата от трите съставки на извличането.

Наред с това самите експерименти откриха един действителен дефект, който преглед на кода не беше открил: безименните параметри не преживяваха пълния цикъл, защото излъчвателят ги записваше без име, а четецът прочиташе двоеточието обратно като част от типа. Дефектът е поправен и е покрит с тест. Това е и най-прекият довод в полза на самия подход: щом и двете посоки са автоматизирани, разминаването става измеримо, а не подозирано, и то важи както за чужд код, така и за самия инструмент.

За по-нататъшна работа се очертават пет посоки. Първата е замяна на последователното търсене при разрешаване на имената с речник, което премахва квадратичното поведение, видимо над 400 класа. Втората е премахване на дублирането на кратността, която днес се пази и върху връзката, и върху атрибута, поради което една промяна поражда две верни съобщения вместо едно. Третата е разширяване на покритието на езика към конструкции, останали извън проверката: приятелски декларации, наследяване от специализация на шаблон и вложени шаблони. Четвъртата е свързване на проверката за разминаване с процеса на непрекъсната интеграция, за което инструментът вече е готов, тъй като завършва с ненулев код при разлика. Петата е допълване на модела с поведенческа информация, което е предпоставка за извличане, а не само за излъчване, на диаграмите на случаите на употреба и на дейностите.

ЛИТЕРАТУРА

- [1] E. J. Chikofsky and J. H. Cross, “Reverse engineering and design recovery: A taxonomy,” *IEEE Software*, vol. 7, no. 1, pp. 13–17, 1990.
- [2] Object Management Group, “OMG unified modeling language (OMG UML), version 2.5.1,” Tech. Rep. formal/2017-12-05, Object Management Group, 2017.
- [3] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*. Addison-Wesley, 2 ed., 2005.
- [4] J. Rumbaugh, I. Jacobson, and G. Booch, *The Unified Modeling Language Reference Manual*. Addison-Wesley, 2 ed., 2004.
- [5] K. Beck and W. Cunningham, “A laboratory for teaching object-oriented thinking,” in *Conference Proceedings on Object-Oriented Programming Systems, Languages and Applications (OOPSLA '89)*, pp. 1–6, 1989.
- [6] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. Addison-Wesley, 3 ed., 2003.
- [7] P. P.-S. Chen, “The entity-relationship model — toward a unified view of data,” *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, 1976.
- [8] S. W. Ambler, *Agile Database Techniques: Effective Strategies for the Agile Software Developer*. John Wiley and Sons, 2003.
- [9] LLVM Project, “Clang c interface to clang (libclang) — documentation.” https://clang.llvm.org/doxygen/group__CINDEX.html. Достъпен онлайн.
- [10] LLVM Project, “Clang compiler user’s manual: AST dumping in JSON format (-Xclang -ast-dump=json).” <https://clang.llvm.org/docs/UsersManual.html>. Достъпен онлайн.
- [11] T. Bray, “The JavaScript object notation (JSON) data interchange format,” Tech. Rep. RFC 8259, Internet Engineering Task Force, 2017.
- [12] D. van Heesch, “Doxygen manual.” <https://www.doxygen.nl/manual/>. Достъпен онлайн.
- [13] J. Ellson, E. Gansner, L. Koutsofios, S. C. North, and G. Woodhull, “Graphviz — open source graph drawing tools,” in *Graph Drawing, LNCS 2265*, pp. 483–484, Springer, 2002.

- [14] A. Roques, “PlantUML language reference guide.” <https://plantuml.com/guide>. Достъпен онлайн.
- [15] K. Sveidqvist and the Mermaid contributors, “Mermaid documentation: Class diagrams.” <https://mermaid.js.org/syntax/classDiagram.html>. Достъпен онлайн.
- [16] ISO/IEC, “ISO/IEC 14882:2020, programming languages — C++,” tech. rep., International Organization for Standardization, 2020.
- [17] Алекс Цветанов, “[**TODO: точното заглавие на дипломната работа за ОКС „Бакалавър“**],” Дипломна работа за ОКС „Бакалавър“, Технически университет — София, Факултет „Компютърни системи и технологии“, [**TODO: година на защита**].

ПРИЛОЖЕНИЯ

Приложение А. Формат на вътрешния модел

Вътрешният модел е записваем във външен вид, за да може да бъде сравняван и съхраняван между отделните стъпки на пълния цикъл. Записът е JSON с два масива на най-горно ниво, `classifiers` и `relations`. Класификаторът носи неквалифицирано и квалифицирано име, вид, признак за абстрактност, шаблонни параметри, стереотипи, етикетирани стойности, атрибути, операции, изброители и мястото в изходния текст, откъдето е дошъл. Атрибутът носи име, тип както е изписан в кода, видимост, обхват, кратност и собствени етикетирани стойности. Операцията носи име, тип на резултата, параметри, видимост, обхват и признаци за виртуалност, чистота и константност. Връзката носи двата края, вид, етикет, кратност, признак за виртуална база и видимост на базата.

Класификаторите се свързват по име, а не по указател. Изборът е продиктуван от изискването моделът да може да бъде записан и прочетен обратно, а също и да бъде сравняван с друг модел по стойност: имената преживяват запис във файл, указателите не.

```
1 // The internal UML model.
2 //
3 // Everything else in the tool either produces this or consumes it. It knows
4 // nothing about C++, nothing about PlantUML or Mermaid and nothing about SQL
5 // ↪ .
6 // That is the whole point of the architecture: a new input language touches
7 // only the reader, a new notation touches only an emitter.
8 //
9 // Classifiers are linked by qualified name, never by pointer. Names survive
10 // serialisation, pointers do not, and the model has to survive a trip
11 // ↪ through
12 // a file for the round-trip experiment to be possible at all.
13
14 #ifndef BLUEPRINT_MODEL_HPP
15 #define BLUEPRINT_MODEL_HPP
16
17 #include <optional>
18 #include <string>
19 #include <vector>
```

```

18 #include "blueprint/json.hpp"
19
20 namespace bp {
21
22 enum class Visibility { Public, Protected, Private, Package };
23
24 // UML calls a static member "classifier scoped" and a normal one "instance
25 // scoped". The model keeps the UML word, not the C++ one.
26 enum class Scope { Instance, Classifier };
27
28 enum class ClassifierKind { Class, Struct, Interface, Enumeration, Template
  ↪ };
29
30 // Relationship kinds of the UML class diagram that this tool infers.
31 enum class RelationKind {
32     Generalization, // inheritance
33     Realization,    // template specialisation of a general template
34     Association,    // reference without ownership
35     Aggregation,    // shared ownership, the part outlives the whole
36     Composition,    // exclusive ownership, the part dies with the whole
37     Dependency      // used in a signature, not held as a member
38 };
39
40 std::string to_string(Visibility v);
41 std::string to_string(Scope s);
42 std::string to_string(ClassifierKind k);
43 std::string to_string(RelationKind r);
44 Visibility visibility_from_string(const std::string& s);
45 Scope scope_from_string(const std::string& s);
46 ClassifierKind classifier_kind_from_string(const std::string& s);
47 RelationKind relation_kind_from_string(const std::string& s);
48
49 // A tagged value in the UML sense: a name and a value attached to an element
  ↪ ,
50 // used here to carry the mapping onto a relational schema.
51 struct TaggedValue {
52     std::string name;
53     std::string value;
54     bool operator==(const TaggedValue&) const = default;

```

```

55 };
56
57 struct Attribute {
58     std::string name;
59     std::string type;           // as written in the source
60     Visibility visibility = Visibility::Private;
61     Scope scope = Scope::Instance;
62     std::string multiplicity = "1";
63     std::vector<TaggedValue> tags;
64     bool operator==(const Attribute&) const = default;
65 };
66
67 struct Parameter {
68     std::string name;
69     std::string type;
70     bool operator==(const Parameter&) const = default;
71 };
72
73 struct Operation {
74     std::string name;
75     std::string return_type;
76     std::vector<Parameter> parameters;
77     Visibility visibility = Visibility::Public;
78     Scope scope = Scope::Instance;
79     bool is_virtual = false;
80     bool is_pure = false;
81     bool is_const = false;
82     bool operator==(const Operation&) const = default;
83
84     // "name(T, U) const" without the return type: the identity of an
85     // operation for overload resolution, and what the checker matches on.
86     std::string signature() const;
87 };
88
89 struct Relation {
90     std::string from;           // qualified name of the source classifier
91     std::string to;             // qualified name of the target classifier
92     RelationKind kind = RelationKind::Association;
93     std::string label;          // member name that produced it, when any

```

```

94     std::string multiplicity = "1"; // at the target end
95     bool is_virtual_base = false; // virtual inheritance
96     Visibility visibility = Visibility::Public; // access of the base, for
    ↪ generalization
97     bool operator==(const Relation&) const = default;
98
99     // Two relations are the same edge when endpoints, kind and label agree.
100     std::string key() const;
101 };
102
103 struct Classifier {
104     std::string name; // unqualified
105     std::string qualified_name; // with namespaces, "::" separated
106     ClassifierKind kind = ClassifierKind::Class;
107     std::vector<std::string> template_parameters;
108     std::vector<std::string> stereotypes;
109     std::vector<TaggedValue> tags;
110     std::vector<Attribute> attributes;
111     std::vector<Operation> operations;
112     std::vector<std::string> enumerators;
113     bool is_abstract = false;
114     std::string source_file;
115     int source_line = 0;
116     bool operator==(const Classifier&) const = default;
117
118     const TaggedValue* tag(const std::string& name) const;
119     bool has_stereotype(const std::string& s) const;
120 };
121
122 struct Model {
123     std::string name = "model";
124     std::vector<Classifier> classifiers;
125     std::vector<Relation> relations;
126
127     const Classifier* find(const std::string& qualified_name) const;
128     Classifier* find(const std::string& qualified_name);
129     bool contains(const std::string& qualified_name) const { return find(
    ↪ qualified_name) != nullptr; }
130

```

```

131 // Sorts classifiers, members and relations into a canonical order so
↳ that
132 // two models built by different paths compare and print identically.
133 void normalize();
134
135 // Number of comparable elements: classifiers, attributes, operations and
136 // relations. The denominator of the round-trip fidelity measure.
137 std::size_t element_count() const;
138
139 Json to_json() const;
140 static Model from_json(const Json& j);
141 };
142
143 } // namespace bp
144
145 #endif

```

Листинг 1: Дефиниция на вътрешния модел, include/blueprint/model.hpp

Приложение Б. Примерен вход и получените от него диаграми

Приложението съдържа малката единица за превод, върху която е проведен пълният цикъл. Тя обхваща наследяване, абстрактен базов клас, асоциация, агрегация, композиция и кратност много, като всяка от тези конструкции се среща в нея точно по веднъж, за да сочи един евентуален отказ към едно правило, а не към сплетени правила.

```

1 // A small annotated domain, used as the forward-engineering example.
2 //
3 // It is deliberately ordinary: a lending library. What matters is that every
4 // construction the reader has to distinguish appears here exactly once, so
5 // that a failure in the generated schema points at one rule rather than at a
6 // tangle of them.
7 //
8 // Author          value member, plain persistent class
9 // Shelf           referenced without ownership, an association
10 // LibraryItem     abstract base of the hierarchy, carries the strategy
11 // Book, Journal   concrete subclasses
12 // Catalogue       owns items exclusively and shares a location, and holds

```

```

13 //          a collection, which is where multiplicity comes from
14 #pragma once
15
16 #include <memory>
17 #include <string>
18 #include <vector>
19
20 #include "blueprint/annotations.hpp"
21
22 namespace library {
23
24 class Author {
25     BP_TABLE("authors");
26     BP_PK(id);
27     BP_COLUMN(name, "VARCHAR(120)");
28
29 public:
30     int id;
31     std::string name;
32     int birth_year;
33 };
34
35 // A shelf exists independently of anything standing on it, so every
36 ↪ reference
37 // to it is an association and never an ownership claim.
38 class Shelf {
39     BP_TABLE("shelves");
40     BP_PK(id);
41     BP_COLUMN(code, "CHAR(8)");
42
43 public:
44     int id;
45     std::string code;
46 };
47
48 // The strategy tag lives on the base class, so a reader of the source can
49 ↪ see
50 // how the hierarchy will be stored without opening the tool.
51 class LibraryItem {

```

```

50     BP_TABLE("items");
51     BP_INHERITANCE("single-table");
52     BP_PK(id);
53     BP_COLUMN(title, "VARCHAR(200)");
54
55 public:
56     virtual ~LibraryItem();
57     virtual double late_fee_per_day() const = 0;
58     virtual bool is_lendable() const;
59
60     int id;
61     std::string title;
62
63 protected:
64     // A value member: the author record lives and dies with the item, which
65     // is composition.
66     Author author;
67     // A pointer without ownership: the shelf outlives the item, association.
68     Shelf* shelf;
69 };
70
71 class Book : public LibraryItem {
72     BP_TABLE("books");
73     BP_COLUMN(isbn, "CHAR(13)");
74
75 public:
76     double late_fee_per_day() const override;
77     std::string isbn;
78     int page_count;
79 };
80
81 class Journal : public LibraryItem {
82     BP_TABLE("journals");
83     BP_COLUMN(issn, "CHAR(9)");
84
85 public:
86     double late_fee_per_day() const override;
87     bool is_lendable() const override;
88     std::string issn;

```

```

89     int volume;
90 };
91
92 // The four ownership spellings side by side. Reading the generated diagram
93 // against this class is the quickest check that the inference still works.
94 class Catalogue {
95     BP_TABLE("catalogues");
96     BP_PK(id);
97
98 public:
99     int id;
100     std::string name;
101
102     void add(std::unique_ptr<LibraryItem> item);
103     Shelf* locate(const std::string& title) const;
104
105 private:
106     std::vector<std::unique_ptr<LibraryItem>> items; // composition, many
107     std::shared_ptr<Shelf> default_shelf;           // aggregation, at most
108     ↪ one
109     Shelf* overflow;                               // association, at most
110     ↪ one
111     Author curator;                                // composition, exactly
112     ↪ one
113 };
114
115 } // namespace library

```

Листинг 2: Анотиран примерен вход, examples/library.hpp

Следват двата текста, получени от този вход. И двата са породени от един и същ модел с blueprint diagram и се различават само по нотацията.

```

1 @startuml
2 ' generated by Blueprint
3 title library
4 skinparam classAttributeIconSize 0
5
6 class "library::Author" as library__Author <<persistent>> {

```

```

7   + birth_year : int
8   + id : int
9   + name : std::string
10  }
11
12  note top of library__Author : bp: table=authors; id.pk=true; name.column-type
    ↪ =VARCHAR(120)
13
14  class "library::Book" as library__Book <<persistent>> {
15      + isbn : std::string
16      + page_count : int
17      + late_fee_per_day() const : double
18  }
19
20  note top of library__Book : bp: table=books; isbn.column-type=CHAR(13)
21
22  class "library::Catalogue" as library__Catalogue <<persistent>> {
23      - curator : Author
24      - default_shelf : std::shared_ptr<library::Shelf> [0..1]
25      + id : int
26      - items : std::vector<std::unique_ptr<library::LibraryItem>> [*]
27      + name : std::string
28      - overflow : Shelf * [0..1]
29      + add(item : std::unique_ptr<library::LibraryItem>) : void
30      + locate(title : const std::string &) const : Shelf *
31  }
32
33  note top of library__Catalogue : bp: table=catalogues; id.pk=true
34
35  class "library::Journal" as library__Journal <<persistent>> {
36      + issn : std::string
37      + volume : int
38      + is_lendable() const : bool
39      + late_fee_per_day() const : double
40  }
41
42  note top of library__Journal : bp: table=journals; issn.column-type=CHAR(9)
43
44  abstract class "library::LibraryItem" as library__LibraryItem <<persistent>>

```

```

↳ {
45   # author : Author
46   + id : int
47   # shelf : Shelf * [0..1]
48   + title : std::string
49   + {virtual} is_lendable() const : bool
50   + {abstract} late_fee_per_day() const : double
51   + {virtual} ~LibraryItem()
52 }
53
54 note top of library__LibraryItem : bp: inheritance=single-table; table=items;
↳ id.pk=true; title.column-type=VARCHAR(200)
55
56 class "library::Shelf" as library__Shelf <<persistent>> {
57   + code : std::string
58   + id : int
59 }
60
61 note top of library__Shelf : bp: table=shelves; code.column-type=CHAR(8); id.
↳ pk=true
62
63 library__LibraryItem <|-- library__Book
64 library__Catalogue o-- "0..1" library__Shelf : default_shelf
65 library__Catalogue --> "0..1" library__Shelf : overflow
66 library__Catalogue *-- "1" library__Author : curator
67 library__Catalogue *-- "*" library__LibraryItem : items
68 library__LibraryItem <|-- library__Journal
69 library__LibraryItem --> "0..1" library__Shelf : shelf
70 library__LibraryItem *-- "1" library__Author : author
71 @enduml

```

Листинг 3: Породена диаграма на класове в PlantUML

```

1 %% generated by Blueprint
2 classDiagram
3   %% title: library
4   direction LR
5   class library__Author["library::Author"] {

```

```

6      <<persistent>>
7      +int birth_year
8      +int id
9      +std::string name
10     }
11     note for library__Author "bp: table=authors; id.pk=true; name.column-type=
↳ VARCHAR(120)"
12     class library__Book["library::Book"] {
13         <<persistent>>
14         +std::string isbn
15         +int page_count
16         +late_fee_per_day() const double
17     }
18     note for library__Book "bp: table=books; isbn.column-type=CHAR(13)"
19     class library__Catalogue["library::Catalogue"] {
20         <<persistent>>
21         -Author curator
22         -std::shared_ptr~library::Shelf~ default_shelf [0..1]
23         +int id
24         -std::vector~std::unique_ptr~library::LibraryItem~~ items [*]
25         +std::string name
26         -Shelf * overflow [0..1]
27         +add(std::unique_ptr~library::LibraryItem~ item) void
28         +locate(const std::string & title) const Shelf *
29     }
30     note for library__Catalogue "bp: table=catalogues; id.pk=true"
31     class library__Journal["library::Journal"] {
32         <<persistent>>
33         +std::string issn
34         +int volume
35         +is_lendable() const bool
36         +late_fee_per_day() const double
37     }
38     note for library__Journal "bp: table=journals; issn.column-type=CHAR(9)"
39     class library__LibraryItem["library::LibraryItem"] {
40         <<abstract>>
41         <<persistent>>
42         #Author author
43         +int id

```

```

44     #Shelf * shelf [0..1]
45     +std::string title
46     +is_lendable() const bool
47     +late_fee_per_day()* const double
48     +~LibraryItem()
49 }
50 note for library__LibraryItem "bp: inheritance=single-table; table=items;
↪ id.pk=true; title.column-type=VARCHAR(200)"
51 class library__Shelf["library::Shelf"] {
52     <<persistent>>
53     +std::string code
54     +int id
55 }
56 note for library__Shelf "bp: table=shelves; code.column-type=CHAR(8); id.pk
↪ =true"
57 library__LibraryItem <|-- library__Book
58 library__Catalogue o-- "0..1" library__Shelf : default_shelf
59 library__Catalogue --> "0..1" library__Shelf : overflow
60 library__Catalogue *-- "1" library__Author : curator
61 library__Catalogue *-- "*" library__LibraryItem : items
62 library__LibraryItem <|-- library__Journal
63 library__LibraryItem --> "0..1" library__Shelf : shelf
64 library__LibraryItem *-- "1" library__Author : author

```

Листинг 4: Породена диаграма на класове в Mermaid

Поведенческите модели се превеждат по същия начин. По-долу е диаграмата на случаите на употреба в PlantUML и диаграмата на дейностите в Mermaid, тоест същите два модела, които Фиг. 2 и Фиг. 3 показват.

```

1 @startuml
2 ' generated by Blueprint
3 left to right direction
4 title Blueprint
5
6 actor "Developer" as DEV
7 actor "Designer" as ARCH
8 actor "Build pipeline" as CI
9

```

```

10 rectangle "Blueprint" {
11     usecase "Extract a model from C++ source" as UC1
12     usecase "Emit a class diagram" as UC2
13     usecase "Generate a relational schema" as UC3
14     usecase "Generate class skeletons from a schema" as UC4
15     usecase "Read a diagram back into a model" as UC5
16     usecase "Report the drift between diagram and code" as UC6
17     usecase "Choose the inheritance strategy" as UC7
18 }
19
20 DEV --> UC1
21 DEV --> UC2
22 DEV --> UC3
23 DEV --> UC4
24 ARCH --> UC2
25 ARCH --> UC5
26 ARCH --> UC6
27 CI --> UC6
28 UC2 ..> UC1 : <<include>>
29 UC3 ..> UC1 : <<include>>
30 UC6 ..> UC1 : <<include>>
31 UC6 ..> UC5 : <<include>>
32 UC7 ..> UC3 : <<extend>>
33 @enduml

```

Листинг 5: Породена диаграма на случаите на употреба в PlantUML

```

1 %% generated by Blueprint
2 flowchart TD
3     %% title: Full round trip
4     S(("S"))
5     A1["Run the compiler front end over the translation unit"]
6     A2["Read the abstract syntax tree into the model"]
7     D1{"Does any classifier carry the persistent stereotype"}
8     A3["Emit the class diagram in both notations"]
9     A4["Derive the relational schema"]
10    A5["Generate the class skeletons from the schema"]
11    A6["Extract a model from the generated skeletons"]

```

```

12  A7["Read the diagram back into a model"]
13  A8["Compare the two models"]
14  D2{"Do the two models agree"}
15  A9["Report the drift and fail the build"]
16  A10["Report the round trip as closed"]
17  E(("E"))
18  S --> A1
19  A1 --> A2
20  A2 --> D1
21  D1 -->|yes| A4
22  D1 -->|no| A3
23  A4 --> A5
24  A5 --> A6
25  A6 --> A8
26  A3 --> A7
27  A7 --> A8
28  A8 --> D2
29  D2 -->|yes| A10
30  D2 -->|no| A9
31  A9 --> E
32  A10 --> E

```

Листинг 6: Породена диаграма на дейностите в Mermaid

Приложение В. Породена релационна схема и обратно породени заготовки

Схемата по-долу е получена от модела на Приложение Б при стратегия *една таблица за йерархията*. Класовете Book и Journal са свити в таблицата items, която получава колона разграничител, а колоните им допускат празна стойност. Асоциацията с кратност много между Catalogue и LibraryItem поражда свързваща таблица. Името на класа пътува в коментар пред всяка таблица, а името на атрибута в коментар след колоната, чието име се различава от неговото, за да не се налага при обратния преход да се отгатва единствено число на английска дума.

```

1  -- generated by Blueprint from model 'library'
2  -- inheritance strategy: single-table

```

```

3
4 -- class: library::Author
5 CREATE TABLE authors (
6     birth_year INTEGER,
7     id INTEGER NOT NULL,
8     name VARCHAR(120),
9     PRIMARY KEY (id)
10 );
11
12 -- class: library::Catalogue
13 CREATE TABLE catalogues (
14     id INTEGER NOT NULL,
15     name TEXT,
16     default_shelf_id INTEGER, -- attribute: default_shelf
17     overflow_id INTEGER, -- attribute: overflow
18     curator_id INTEGER NOT NULL, -- attribute: curator
19     PRIMARY KEY (id),
20     FOREIGN KEY (default_shelf_id) REFERENCES shelves(id),
21     FOREIGN KEY (overflow_id) REFERENCES shelves(id),
22     FOREIGN KEY (curator_id) REFERENCES authors(id)
23 );
24
25 -- class: library::LibraryItem
26 CREATE TABLE items (
27     id INTEGER NOT NULL,
28     title VARCHAR(200),
29     class_type TEXT NOT NULL,
30     isbn CHAR(13),
31     page_count INTEGER,
32     issn CHAR(9),
33     volume INTEGER,
34     shelf_id INTEGER, -- attribute: shelf
35     author_id INTEGER NOT NULL, -- attribute: author
36     PRIMARY KEY (id),
37     FOREIGN KEY (shelf_id) REFERENCES shelves(id),
38     FOREIGN KEY (author_id) REFERENCES authors(id)
39 );
40
41 -- class: library::Shelf

```

```

42 CREATE TABLE shelves (
43     code CHAR(8),
44     id INTEGER NOT NULL,
45     PRIMARY KEY (id)
46 );
47
48 -- class: library::Catalogue (join)
49 CREATE TABLE catalogues_items (
50     catalogue_id INTEGER NOT NULL,
51     items_id INTEGER NOT NULL,
52     FOREIGN KEY (catalogue_id) REFERENCES catalogues(id),
53     FOREIGN KEY (items_id) REFERENCES items(id)
54 );

```

Листинг 7: Породена схема при стратегия „една таблица за йерархията“

Обратният преход от схема към код затваря цикъла. Породеният заглавен файл съдържа само декларации и носи макросите за аотиране, така че повторното извличане от него дава съпоставим модел. Че той е действителен C++, се проверява в демонстрацията с `clang -fsyntax-only`, а не се твърди.

```

1  // Generated by Blueprint from schema 'library'.
2  // Inheritance strategy of the source schema: single-table.
3  //
4  // Declarations only. Blueprint never writes an operation body and never
5  // overwrites a file that already holds one.
6  #pragma once
7
8  #include <string>
9
10 #include "blueprint/annotations.hpp"
11
12 namespace library {
13
14 class Author {
15     BP_TABLE("authors");
16     BP_COLUMN(birth_year, "INTEGER");
17     BP_PK(id);

```

```

18     BP_COLUMN(id, "INTEGER");
19     BP_COLUMN(name, "VARCHAR(120)");
20
21 public:
22     int birth_year;
23     int id;
24     std::string name;
25 };
26
27 class Catalogue {
28     BP_TABLE("catalogues");
29     BP_PK(id);
30     BP_COLUMN(id, "INTEGER");
31     BP_COLUMN(name, "TEXT");
32     BP_COLUMN(default_shelf, "INTEGER");
33     BP_COLUMN(overflow, "INTEGER");
34     BP_COLUMN(curator, "INTEGER");
35
36 public:
37     int id;
38     std::string name;
39     int default_shelf; // foreign key onto shelves(id)
40     int overflow; // foreign key onto shelves(id)
41     int curator; // foreign key onto authors(id)
42 };
43
44 class LibraryItem {
45     BP_TABLE("items");
46     BP_PK(id);
47     BP_COLUMN(id, "INTEGER");
48     BP_COLUMN(title, "VARCHAR(200)");
49     BP_COLUMN(class_type, "TEXT");
50     BP_COLUMN(isbn, "CHAR(13)");
51     BP_COLUMN(page_count, "INTEGER");
52     BP_COLUMN(issn, "CHAR(9)");
53     BP_COLUMN(volume, "INTEGER");
54     BP_COLUMN(shelf, "INTEGER");
55     BP_COLUMN(author, "INTEGER");
56

```

```

57 public:
58     int id;
59     std::string title;
60     std::string class_type;
61     std::string isbn;
62     int page_count;
63     std::string issn;
64     int volume;
65     int shelf; // foreign key onto shelves(id)
66     int author; // foreign key onto authors(id)
67 };
68
69 class Shelf {
70     BP_TABLE("shelves");
71     BP_COLUMN(code, "CHAR(8)");
72     BP_PK(id);
73     BP_COLUMN(id, "INTEGER");
74
75 public:
76     std::string code;
77     int id;
78 };
79
80 } // namespace library

```

Листинг 8: Заготовки на класове, породени обратно от схемата

Приложение Г. Ключови места от изходния текст

Пълният изходен текст е в хранилището. Тук се вмъкват двете места, на които се взема същинското решение, а не се пренася информация.

Първото е правилото, което превръща изписването на един член във вид на връзка. То е съзнателно събрано в една функция, за да може да бъде прочетено наведнъж и оспорено наведнъж.

```

133 // The ownership rule of the tool, in one function.
134 //

```

```

135 // T, std::optional<T>, std::unique_ptr<T>    exclusive ownership
    ↪ composition
136 // std::shared_ptr<T>                        shared ownership
    ↪ aggregation
137 // T*, T&                                    no ownership
    ↪ association
138 // container of the above                    the same, with *
139 //
140 // The reading follows UML: composition means the part dies with the whole,
    ↪ so
141 // a member held by value or by a pointer that owns it exclusively is a
142 // composition; a shared_ptr keeps the part alive past the whole, which is
143 // aggregation; and a raw pointer or reference asserts nothing about lifetime
    ↪ ,
144 // which leaves a plain association.
145 MemberClassification classify_member_type(const std::string& type,
146                                         const std::vector<std::string>&
    ↪ known) {
147     MemberClassification result;
148     std::string t = strip_cv(type);
149
150     // References and raw pointers first: they bind tighter than any template
151     // spelling around them.
152     if (ends_with(t, "&&") || ends_with(t, "&")) {
153         t = strip_cv(t.substr(0, t.size() - (ends_with(t, "&&") ? 2 : 1)));
154         const std::string target = resolve_name(t, known);
155         if (target.empty()) return result;
156         result = {true, RelationKind::Association, target, "1"};
157         return result;
158     }
159     if (ends_with(t, "*")) {
160         t = strip_cv(t.substr(0, t.size() - 1));
161         const std::string target = resolve_name(t, known);
162         if (target.empty()) return result;
163         result = {true, RelationKind::Association, target, "0..1"};
164         return result;
165     }
166
167     std::string base;

```

```

168     std::string args;
169     if (split_template(t, base, args)) {
170         const std::string tmp1 = last_component(base);
171         const std::vector<std::string> parts = split_template_args(args);
172         if (parts.empty()) return result;
173
174         if (owning_smart_pointers().count(tmp1)) {
175             MemberClassification inner = classify_member_type(parts[0], known
↪ );
176             if (!inner.is_relation) return result;
177             // The pointer owns, so the inner value kind does not weaken it,
178             // but a shared inner does: unique_ptr<shared_ptr<T>> is shared.
179             inner.kind = inner.kind == RelationKind::Aggregation ?
↪ RelationKind::Aggregation
180                                     :
↪ RelationKind::Composition;
181             if (inner.kind == RelationKind::Composition &&
182                 classify_member_type(parts[0], known).kind == RelationKind::
↪ Association) {
183                 inner.kind = RelationKind::Association;
184             }
185             inner.multiplicity = "0..1";
186             return inner;
187         }
188         if (shared_smart_pointers().count(tmp1)) {
189             MemberClassification inner = classify_member_type(parts[0], known
↪ );
190             if (!inner.is_relation) return result;
191             inner.kind = RelationKind::Aggregation;
192             inner.multiplicity = "0..1";
193             return inner;
194         }
195         if (sequence_containers().count(tmp1) || map_containers().count(tmp1)
↪ ) {
196             // For a map the modelled end is the mapped type, not the key.
197             const std::size_t index = map_containers().count(tmp1) && parts.
↪ size() > 1 ? 1 : 0;
198             MemberClassification inner = classify_member_type(parts[index],
↪ known);

```

```

199         if (!inner.is_relation) return result;
200         inner.multiplicity = "*";
201         return inner;
202     }
203     // Any other template used by value is a value member of that
↪ template.
204     const std::string target = resolve_name(base, known);
205     if (target.empty()) return result;
206     return {true, RelationKind::Composition, target, "1"};
207 }
208
209 const std::string target = resolve_name(t, known);
210 if (target.empty()) return result;
211 return {true, RelationKind::Composition, target, "1"};
212 }

```

Листинг 9: Правилото за собствеността, `src/reader.cpp`

Второто е броенето на запазените елементи, което е числителят на дела от Табл. 8. То се брои поелементно, а не се извежда от броя на разликите, и коментарът в кода казва защо.

```

116 std::size_t preserved_elements(const Model& code, const Model& returned) {
117     std::size_t kept = 0;
118     for (const auto& c : code.classifiers) {
119         const Classifier* other = returned.find(c.qualified_name);
120         if (!other) continue;
121         if (c.kind == other->kind && c.is_abstract == other->is_abstract) ++
↪ kept;
122         for (const auto& a : c.attributes) {
123             for (const auto& b : other->attributes) {
124                 if (a.name == b.name && describe(a) == describe(b)) { ++kept;
↪ break; }
125             }
126         }
127         for (const auto& o : c.operations) {
128             for (const auto& p : other->operations) {
129                 if (o.signature() == p.signature() && describe(o) == describe
↪ (p)) { ++kept; break; }

```

```

130         }
131     }
132 }
133 for (const auto& r : code.relations) {
134     for (const auto& s : returned.relations) {
135         if (relation_identity(r) == relation_identity(s) && describe(r)
↵ == describe(s)) {
136             ++kept;
137             break;
138         }
139     }
140 }
141 return kept;
142 }

```

Листинг 10: Броене на запазените елементи, `src/diff.cpp`